Software Engineering

Lecture 03 — Agile Software Development — Question Bank

Part 1: Multiple Choice (25)

1. According to the chapter, what is the most critical requirement for most business systems?

حسب الفصل، ما هو أهم متطلب لمعظم أنظمة الأعمال؟

A. Complete and stable requirements
B. Rapid software development and delivery
C. Detailed design documentation
D. Minimal use of development tools

✅ **الإجابة الصحيحة: B. Rapid software development and delivery**

💡 ليه صح: الشركات تعمل في بيئة سريعة التغير فتحتاج برمجيات جديدة بسرعة لاستغلال الفرص، وقد تتنازل أحيانًا عن بعض الجودة أو المتطلبات مقابل التسليم السريع. وجود متطلبات كاملة ومستقرة شبه مستحيل في هذه البيئة.

📍 المرجع: Lecture 03 — Chapter introduction

2. For which type of software is a plan-driven approach still the right one, according to the chapter?

لأي نوع من البرمجيات يظل النهج المبني على الخطة (plan-driven) هو الأنسب حسب الفصل؟

A. Small mobile apps
B. Short-lived marketing websites
C. Prototypes built for demonstrations
D. Safety-critical control systems where a complete analysis is essential

✅ **الإجابة الصحيحة: D. Safety-critical control systems where a complete analysis is essential**

💡 ليه صح: الأنظمة الحرجة للسلامة مثل أنظمة التحكم تحتاج تحليلًا كاملًا للنظام، فالـ plan-driven مناسب لها. أما في بيئة الأعمال السريعة فقد يسبب مشاكل حقيقية.

📍 المرجع: Lecture 03 — Chapter introduction

3. Which of the following is a common characteristic of all agile methods?

أي مما يلي خاصية مشتركة بين كل طرق Agile؟

A. Specification, design and implementation are interleaved
B. A complete detailed specification is written before any coding
C. Testing is done only after the whole system is built
D. A new release is made once a year

✅ **الإجابة الصحيحة: A. Specification, design and implementation are interleaved**

💡 ليه صح: في Agile عمليات المواصفات والتصميم والتنفيذ متداخلة ولا توجد مواصفات تفصيلية. الاختيارات الأخرى تصف نهجًا تقليديًا، كما أن الإصدارات في Agile تكون كل أسبوعين أو ثلاثة.

📍 المرجع: Lecture 03 — Chapter introduction

4. How does iteration differ between plan-driven and agile development (Figure 3.1)?

كيف يختلف التكرار (iteration) بين التطوير plan-driven وAgile (الشكل 3.1)؟

A. Plan-driven iterates across activities and agile iterates within activities
B. Neither approach iterates
C. In plan-driven development iteration occurs within activities, while in agile it occurs across activities
D. Agile uses formal documents between stages while plan-driven does not

✅ الإجابة الصحيحة: C. In plan-driven development iteration occurs within activities, while in agile it occurs across activities

💡 ليه صح: في plan-driven يحدث التكرار داخل كل نشاط وتُستخدم مستندات رسمية للتواصل بين المراحل، أما في Agile فيحدث عبر الأنشطة فتُطوَّر المتطلبات والتصميم معًا. الاختيار الأول معكوس والرابع معكوس أيضًا.

📍 المرجع: Lecture 03 — Chapter introduction (Figure 3.1)

5. According to the agile manifesto, which item is valued more than following a plan?

حسب بيان Agile (manifesto)، أي عنصر يُقدَّر أكثر من اتباع الخطة؟

A. Comprehensive documentation
B. Responding to change
C. Contract negotiation
D. Processes and tools

✅ الإجابة الصحيحة: B. Responding to change

💡 ليه صح: القيم الأربع: الأفراد والتفاعلات على العمليات والأدوات، والبرمجيات العاملة على التوثيق الشامل، وتعاون العميل على التفاوض على العقد، والاستجابة للتغيير على اتباع الخطة. بقية الاختيارات مذكورة كعناصر أقل قيمة.

📍 المرجع: Lecture 03 — 3.1 Agile methods

6. Which agile principle says that the skills of the team should be exploited and team members should develop their own ways of working?

أي مبدأ من مبادئ Agile ينص على استغلال مهارات الفريق وترك أعضائه يطورون طرق عملهم بأنفسهم؟

A. People, not process
B. Incremental delivery
C. Maintain simplicity
D. Embrace change

✅ الإجابة الصحيحة: A. People, not process

💡 ليه صح: مبدأ People, not process يعترف بمهارات الفريق ويتجنب العمليات الإلزامية. Incremental delivery عن التسليم على دفعات، وMaintain simplicity عن البساطة، وEmbrace change عن توقع تغيّر المتطلبات.

📍 المرجع: Lecture 03 — 3.1 Agile methods (Figure 3.2)

7. For which kinds of system development have agile methods been particularly successful?

في أي أنواع تطوير الأنظمة نجحت طرق Agile بشكل خاص؟

A. Large systems built by many geographically dispersed teams
B. Safety-critical systems with heavy regulation
C. Systems tightly integrated with other systems built in parallel
D. Small or medium-sized product development, and custom systems with a committed customer and few external stakeholders

✅ الإجابة الصحيحة: D. Small or medium-sized product development, and custom systems with a committed customer and few external stakeholders

💡 ليه صح: نجحت في تطوير المنتجات الصغيرة والمتوسطة، وفي الأنظمة المخصصة داخل مؤسسة حيث العميل ملتزم بالمشاركة وعدد الجهات الخارجية والقوانين قليل. الاختيارات الأخرى بيئات تحتاج تنسيقًا وخطة.

📍 المرجع: Lecture 03 — 3.1 Agile methods

8. Why was the name 'Extreme Programming' (XP) chosen?

لماذا سُمّيت 'Extreme Programming' (XP) بهذا الاسم؟

A. It was developed for extreme sports software
B. It requires programmers to work extreme hours
C. It pushes recognized good practices, such as iterative development, to 'extreme' levels
D. It uses the largest possible amount of documentation

✅ الإجابة الصحيحة: C. It pushes recognized good practices, such as iterative development, to 'extreme' levels

💡 ليه صح: الاسم صاغه Kent Beck لأن النهج دفع الممارسات الجيدة المعروفة مثل التطوير التكراري إلى مستويات 'متطرفة'. وهو يرفض العمل لساعات طويلة (Sustainable pace)، فالاختيار الثاني خطأ.

📍 المرجع: Lecture 03 — 3.2 Agile development techniques

9. In XP, what does the practice 'collective ownership' mean?

في XP، ماذا تعني ممارسة 'collective ownership'؟

A. Only the senior developer owns the core code
B. Pairs of developers work on all areas of the system, so anyone can change anything and everyone takes responsibility for all the code
C. Each developer owns one module exclusively
D. The customer owns the source code

✅ الإجابة الصحيحة: B. Pairs of developers work on all areas of the system, so anyone can change anything and everyone takes responsibility for all the code

💡 ليه صح: الأزواج يعملون على كل أجزاء النظام فلا تتكون جزر خبرة ويتحمل الجميع مسؤولية كل الكود. هذا عكس ملكية كل مبرمج لوحدة واحدة.

📍 المرجع: Lecture 03 — 3.2 Agile development techniques (Figure 3.4)

10. Which XP practice requires that, once finished work is integrated into the whole system, all the unit tests in the system must pass?

أي ممارسة في XP تتطلب أنه بعد دمج العمل المنتهي في النظام يجب أن تنجح كل اختبارات الوحدة؟

A. Continuous integration
B. Sustainable pace
C. Small releases
D. Simple design

✅ الإجابة الصحيحة: A. Continuous integration

💡 ليه صح: Continuous integration: بمجرد انتهاء المهمة تُدمج في النظام كله ويجب أن تنجح كل unit tests بعد كل دمج. Small releases عن الإصدارات المتكررة، وSimple design عن التصميم بقدر الحاجة فقط.

📍 المرجع: Lecture 03 — 3.2 Agile development techniques (Figure 3.4)

11. Why does XP consider large amounts of overtime unacceptable (sustainable pace)?

لماذا تعتبر XP العمل الإضافي الكبير غير مقبول (sustainable pace)؟

A. It increases the cost of development tools
B. It violates the customer contract
C. The net effect is often to reduce code quality and medium-term productivity
D. It makes pair programming impossible

✅ الإجابة الصحيحة: C. The net effect is often to reduce code quality and medium-term productivity

💡 ليه صح: الفصل يوضح أن الأثر الصافي للإفراط في العمل غالبًا هو انخفاض جودة الكود والإنتاجية على المدى المتوسط. باقي الأسباب غير مذكورة.

📍 المرجع: Lecture 03 — 3.2 Agile development techniques (Figure 3.4)

12. What is the principal problem with user stories mentioned in the chapter?

ما المشكلة الرئيسية في قصص المستخدم (user stories) المذكورة في الفصل؟

A. Customers cannot understand them
B. They require a formal specification language
C. They cannot be broken into tasks
D. Completeness: it is difficult to judge whether enough stories have been written to cover all essential requirements

✅ الإجابة الصحيحة: D. Completeness: it is difficult to judge whether enough stories have been written to cover all essential requirements

💡 ليه صح: المشكلة هي الاكتمال: يصعب الحكم إن كانت القصص تغطي كل المتطلبات الأساسية، وقد لا تعطي قصة واحدة صورة حقيقية للنشاط لأن المستخدم الخبير يترك تفاصيل. على العكس، الناس يربطون بالقصص بسهولة وتُقسَّم إلى tasks.

📍 المرجع: Lecture 03 — 3.2.1 User stories

13. What is refactoring in XP?

ما هو الـ refactoring في XP؟

A. Rewriting the whole system at the end of the project
B. Continuously looking for and implementing code improvements to keep the code simple and avoid structural degradation
C. Adding generality to the code to anticipate future changes
D. Removing all tests after a release

✅ الإجابة الصحيحة: B. Continuously looking for and implementing code improvements to keep the code simple and avoid structural degradation

💡 ليه صح: هو البحث المستمر عن تحسينات للكود وتنفيذها فورًا حتى لو لم تكن هناك حاجة عاجلة، لتجنب تدهور بنية البرنامج مع التغييرات المتتالية. XP ترفض 'التصميم للتغيير' بإضافة العمومية مسبقًا.

📍 المرجع: Lecture 03 — 3.2.2 Refactoring

14. What does test-first development mean?

ماذا يعني التطوير بكتابة الاختبارات أولًا (test-first development)؟

A. Writing automated tests for new functionality before implementing that functionality
B. Testing the system only after a code review
C. Letting an external team write the tests later
D. Skipping unit tests for simple tasks

✅ الإجابة الصحيحة: A. Writing automated tests for new functionality before implementing that functionality

💡 ليه صح: تُكتب الاختبارات (باستخدام إطار اختبار آلي مثل JUnit) قبل كتابة الكود، فتُعرّف الواجهة وسلوك الوظيفة وتقل سوء الفهم. الاختيارات الأخرى تصف اختبارًا متأخرًا أو خارجيًا.

📍 المرجع: Lecture 03 — 3.2.3 Test-first development

15. What is the problem of 'test-lag' in testing?

ما هي مشكلة 'test-lag' في الاختبار؟

A. Automated tests run too slowly
B. Tests fail at random
C. The developer works faster than the tester, so implementation gets ahead of testing and tests tend to be skipped
D. The customer delays writing acceptance tests

✅ الإجابة الصحيحة: C. The developer works faster than the tester, so implementation gets ahead of testing and tests tend to be skipped

💡 ليه صح: تحدث حين يعمل المطوّر أسرع من المختبِر فيتقدم التنفيذ على الاختبار ويميل الفريق لتخطي الاختبارات للحفاظ على الجدول. الـ test-first يتفادى ذلك.

📍 المرجع: Lecture 03 — 3.2.3 Test-first development

16. Why does pair programming work as an informal review process?

لماذا يعمل pair programming كعملية مراجعة غير رسمية؟

A. A manager checks every pair at the end of the day
B. Pairs stay the same for the whole project
C. Only one developer types the code
D. Each line of code is looked at by at least two people

✅ **الإجابة الصحيحة: D. Each line of code is looked at by at least two people**

💡 ليه صح: لأن كل سطر كود يراه شخصان على الأقل. هو أرخص وأسهل تنظيمًا من المراجعات الرسمية (code inspections) لكنه غالبًا يكتشف أخطاء أقل منها. الأزواج في XP تتغير ديناميكيًا.

📍 المرجع: Lecture 03 — 3.2.4 Pair programming

17. What did the studies with more experienced programmers (Arisholm et al.) find about pair programming?

ماذا وجدت الدراسات على مبرمجين أكثر خبرة (Arisholm وزملاؤه) عن pair programming؟

A. A significant loss of productivity compared with two programmers working alone, with some quality benefits that did not fully compensate
B. Productivity was doubled
C. No difference in productivity or quality
D. Pair programming produced more errors

✅ **الإجابة الصحيحة: A. A significant loss of productivity compared with two programmers working alone, with some quality benefits that did not fully compensate**

💡 ليه صح: وجدوا فقدانًا ملحوظًا في الإنتاجية مقارنة بمبرمجين يعملان منفردين، مع فوائد جودة لم تعوّض الكلفة بالكامل. دراسة Williams على الطلاب وجدت إنتاجية مماثلة. ومع ذلك مشاركة المعرفة تقلل المخاطر عند ترك أعضاء الفريق.

📍 المرجع: Lecture 03 — 3.2.4 Pair programming

18. What is the recommended maximum size of a Scrum development team?

ما الحد الأقصى الموصى به لحجم فريق التطوير في Scrum؟

A. Three people
B. Seven people
C. Twelve people
D. Twenty people

✅ **الإجابة الصحيحة: B. Seven people**

💡 ليه صح: Development team: مجموعة منظِّمة لنفسها لا تزيد عن سبعة أشخاص.

📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

19. How long is a Scrum sprint usually?

كم تستغرق دورة Sprint عادةً في Scrum؟

A. 1 to 3 days
B. 1 to 2 months
C. 2 to 4 weeks
D. 6 to 12 months

✅ **الإجابة الصحيحة: C. 2 to 4 weeks**
💡 ليه صح: الـ Sprint هو تكرار تطوير (development iteration) مدته عادةً من 2 إلى 4 أسابيع وبطول ثابت.
📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

20. What is the job of the Product Owner in Scrum?

ما دور الـ Product Owner في Scrum؟

A. Making sure the Scrum process is followed and shielding the team from outside interference
B. Writing all the code of the product
C. Deciding top-down who works on which task
D. Identifying product features or requirements, prioritizing them, and continuously reviewing the product backlog

✅ **الإجابة الصحيحة: D. Identifying product features or requirements, prioritizing them, and continuously reviewing the product backlog**
💡 ليه صح: الـ Product Owner يحدد الميزات أو المتطلبات ويرتبها حسب الأولوية ويراجع الـ product backlog باستمرار. الاختيار الأول دور الـ ScrumMaster، والثالث غير صحيح لأن الفريق ينظّم نفسه.
📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

21. What is the product backlog in Scrum?

ما هو الـ product backlog في Scrum؟

A. A list of 'to do' items the Scrum team must tackle, such as features, requirements, user stories or supplementary tasks
B. A list of bugs found after the release
C. Only the work selected for the current sprint
D. The minutes of the daily Scrum

✅ **الإجابة الصحيحة: A. A list of 'to do' items the Scrum team must tackle, such as features, requirements, user stories or supplementary tasks**
💡 ليه صح: هو قائمة بنود العمل التي يجب أن يتناولها الفريق: ميزات، متطلبات، user stories أو مهام مساعدة مثل تعريف المعمارية أو التوثيق. أما العمل المختار للـ sprint فيسمى sprint backlog.
📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

22. What is 'velocity' in Scrum?

ما هي الـ 'velocity' في Scrum؟

A. The speed of the team's network connection
B. An estimate of how much product backlog effort a team can cover in a single sprint
C. The number of developers in the team
D. The date of the final delivery

✅ الإجابة الصحيحة: B. An estimate of how much product backlog effort a team can cover in a single sprint

💡 ليه صح: هي تقدير لحجم جهد الـ product backlog الذي يستطيع الفريق إنجازه في sprint واحد، وتُستخدم لتقدير ما يمكن تغطيته في الـ sprint القادم ولقياس تحسن الأداء.

📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

23. What happens to items that cannot be completed within a sprint?

ماذا يحدث للبنود التي لا يمكن إنجازها داخل الـ sprint؟

A. The sprint is extended until they are finished
B. They are deleted from the project
C. They are returned to the product backlog, because sprints are never extended
D. They are handed to the ScrumMaster

✅ الإجابة الصحيحة: C. They are returned to the product backlog, because sprints are never extended

💡 ليه صح: لا تُمدَّد الـ sprints أبدًا لاستيعاب العمل غير المنتهي، وتُعاد البنود إلى الـ product backlog.

📍 المرجع: Lecture 03 — 3.3 Agile project management

24. Which statement about the ScrumMaster is correct?

أي عبارة عن الـ ScrumMaster صحيحة؟

A. He or she prioritizes the product backlog
B. He or she writes the sprint backlog alone
C. He or she directs the team top-down
D. He or she ensures the Scrum process is followed and interfaces with the rest of the company, and Scrum developers insist this is not a project manager

✅ الإجابة الصحيحة: D. He or she ensures the Scrum process is followed and interfaces with the rest of the company, and Scrum developers insist this is not a project manager

💡 ليه صح: الـ ScrumMaster مسؤول عن اتباع عملية Scrum وإرشاد الفريق والتعامل مع باقي الشركة وحماية الفريق من التشتت، ومطورو Scrum يؤكدون أنه ليس project manager. ترتيب الـ backlog من مهام الـ Product Owner، ولا يوجد توجيه من أعلى لأسفل.

📍 المرجع: Lecture 03 — 3.3 Agile project management (Figure 3.8)

25. What is the purpose of the daily Scrum meeting?

ما الغرض من اجتماع الـ Scrum اليومي؟

A. A short meeting to review progress and prioritize the work to be done that day
B. A formal review presented to senior management
C. Planning the next product release
D. Writing the project documentation

✅ الإجابة الصحيحة: A. A short meeting to review progress and prioritize the work to be done that day

💡 ليه صح: هو اجتماع قصير (يُفضَّل وجهًا لوجه ويشمل كل الفريق) لمراجعة التقدم وترتيب عمل اليوم: ماذا أنجز كل فرد وما المشاكل وما المخطط للغد. الاختيارات الأخرى ليست غرضه.

📍 المرجع: Lecture 03 — 3.3 Agile project management

Part 2: Essay (10)

Essay 1. Compare plan-driven and agile development, explaining why agile methods appeared.

قارن بين التطوير plan-driven وAgile مع شرح سبب ظهور طرق Agile.

Model answer:

Plan-driven development separates the process into stages with outputs (for example a requirements specification) used to plan the next activity. Iteration occurs within activities and formal documents communicate between stages.

Agile treats design and implementation as the central activities and incorporates requirements elicitation and testing into them. Iteration occurs across activities, so requirements and design are developed together.

Plan-driven overhead is justified for large, critical, long-lived systems built by multiple teams, but for small and medium business systems the overhead dominates and requirements changes force rework.

Dissatisfaction with heavyweight approaches led to agile methods in the late 1990s (Extreme Programming, Scrum, DSDM), which focus on delivering working software quickly and avoiding documentation that will never be used.

In practice the two can be mixed: plan-driven processes may use agile practices and agile methods may include some planned activities.

📍 المرجع: Lecture 03 — Chapter introduction and 3.1

Essay 2. State the agile manifesto and the principles of agile methods.

اذكر بيان Agile (manifesto) ومبادئ طرق Agile.

Model answer:

The manifesto values: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan. The items on the right still have value, but those on the left are valued more.

Principles (Figure 3.2): customer involvement (customers provide and prioritize requirements and evaluate iterations); embrace change (expect requirements to change); incremental delivery (customer specifies the requirements of each increment); maintain simplicity (in the software and in the process); people, not process (exploit team skills and let the team choose its own ways of working).

📍 المرجع: Lecture 03 — 3.1 Agile methods

Essay 3. What are the common characteristics of agile methods and in which situations do they work well?

ما الخصائص المشتركة لطرق Agile وفي أي مواقف تعمل جيدًا؟

Model answer:

Common characteristics: (1) specification, design and implementation are interleaved, with no detailed specification and minimal design documentation; (2) the system is developed in a series of increments, with end-users and stakeholders involved in specifying and evaluating each increment; (3) extensive tool support (automated testing, configuration management, integration, UI production tools).

Increments are small, with new releases typically every two or three weeks, and informal communication is used instead of formal documents.

They work well for product development of small or medium-sized products, and for custom systems within an organization with a committed customer and few external stakeholders or regulations.

They work because the customer and team can communicate continuously, the system is stand-alone, and co-located teams can use informal communication.

📍 المرجع: Lecture 03 — Chapter introduction and 3.1

Essay 4. List and explain the main Extreme Programming practices.

اذكر واشرح أهم ممارسات Extreme Programming.

Model answer:

Incremental planning: requirements are recorded on story cards and the stories for a release are chosen by time available and priority; developers break them into tasks.

Small releases: the minimal useful set of functionality is released first and frequent releases add functionality.

Simple design: enough design to meet current requirements and no more. Test-first development: automated unit tests are written before the functionality.

Refactoring: continuous code improvement to keep code simple. Pair programming: developers work in pairs and check each other's work.

Collective ownership: anyone can change anything. Continuous integration: finished work is integrated at once and all unit tests must pass.

Sustainable pace: large overtime is not acceptable. On-site customer: a customer representative is available full time and defines acceptance tests.

📍 المرجع: Lecture 03 — 3.2 Agile development techniques (Figure 3.4)

Essay 5. Explain user stories: how they are used in planning, and their advantages and problems.

اشرح قصص المستخدم (user stories): كيف تُستخدم في التخطيط ومميزاتها ومشاكلها.

Model answer:

A user story is a scenario of use that might be experienced by a system user. The customer works with the team to write a story card that briefly describes it; the example in the chapter is the 'Prescribing medication' story for the Mentcare system.

The team breaks stories into tasks, estimates effort, and the customer prioritizes the stories for the next release, aiming for functionality that can be implemented in about two weeks.

Advantage: people find stories easier to relate to than conventional requirements documents or use cases, and they help involve users in suggesting requirements.

Problem: completeness. It is hard to judge whether enough stories cover all essential requirements or whether a single story gives a true picture, because experienced users leave things out.

📍 المرجع: Lecture 03 — 3.2.1 User stories

Essay 6. Explain refactoring: why XP uses it and what its limitations are.

اشرح الـ refactoring: لماذا تستخدمه XP وما حدوده.

Model answer:

Traditional software engineering says to design for change. XP discards this because anticipated changes often never materialize, so adding generality is wasted effort.

Instead the code is refactored constantly: the team looks for improvements and implements them immediately, even without an immediate need.

Reason: local changes during incremental development degrade the software structure (duplicated code, inappropriate reuse of parts). Refactoring improves structure and readability. Examples: reorganizing a class hierarchy to remove duplication, renaming attributes and methods, replacing similar code with library calls.

Limitations: development pressure may delay refactoring, and some changes cannot be handled by code-level refactoring and require architecture changes.

📍 المرجع: Lecture 03 — 3.2.2 Refactoring

Essay 7. Explain test-first development and the problems of ensuring complete test coverage.

اشرح الـ test-first development ومشاكل ضمان تغطية اختبارات كاملة.

Model answer:

Test-first development: instead of writing code and then tests, tests are written before the code, so problems are discovered during development. Writing tests implicitly defines an interface and specification of behavior, reducing misunderstandings.

Key features in XP: test-first development, incremental test development from scenarios, user involvement in test development and validation, and automated testing frameworks such as JUnit. Each task generates one or more unit tests, and the customer helps develop acceptance tests.

It avoids test-lag, where implementation gets ahead of testing.

Coverage problems: programmers prefer programming and may write incomplete tests; some tests (for example display logic and screen workflow in a complex UI) are hard to write incrementally; a large set of tests may give a false impression of completeness and crucial parts may stay untested.

📍 المرجع: Lecture 03 — 3.2.3 Test-first development

Essay 8. Discuss the advantages of pair programming and the results of studies on its productivity.

ناقش مميزات pair programming ونتائج الدراسات عن إنتاجيته.

Model answer:

Pair programming: two programmers sit at the same computer, and pairs are created dynamically so all team members work with each other.

Advantages: (1) it supports collective ownership and responsibility (egoless programming); (2) it acts as an informal review process since each line is seen by at least two people, cheaper and easier than formal inspections though it finds fewer errors; (3) it encourages refactoring because others benefit immediately.

Studies: with student volunteers (Williams et al.) productivity was comparable to two people working independently, because pairs discuss before development and have fewer false starts and less rework. Studies with experienced programmers (Arisholm et al.) found a significant loss of productivity, with quality benefits that did not fully compensate.

Knowledge sharing reduces project risk when team members leave, which may by itself make it worthwhile.

📍 المرجع: Lecture 03 — 3.2.4 Pair programming

Essay 9. Describe the Scrum sprint cycle and the main Scrum roles and terms, and mention the benefits users report.

صِف دورة الـ sprint في Scrum وأهم الأدوار والمصطلحات والفوائد التي يذكرها المستخدمون.

Model answer:

Scrum is a framework for organizing agile projects that follows the agile manifesto but does not mandate specific practices such as pair programming or test-first development. It gives some external visibility of what is going on.

Cycle: the input is the product backlog. At the start of each sprint (2 to 4 weeks, fixed length) the Product Owner prioritizes items; the team selects what it can complete using velocity, creating the sprint backlog, and self-organizes. During the sprint the team holds daily Scrums, optionally using a Scrum board. At the end a review meeting improves the process and gives input to the next backlog review. Each sprint produces a potentially shippable product increment. Unfinished items return to the backlog.

Roles: development team (self-organizing, at most seven people), Product Owner (prioritizes backlog), ScrumMaster (ensures the process is followed and interfaces with the company; not formally a project manager).

Benefits: manageable chunks, unstable requirements do not hold up progress, team visibility improves communication and morale, customers see on-time increments and give feedback, and trust between customers and developers is established.

📍 المرجع: Lecture 03 — 3.3 Agile project management

Essay 10. Explain the issues of scaling agile methods and how Scrum is adapted for distributed development.

اشرح مشاكل توسيع طرق Agile وكيف يُكيَّف Scrum للتطوير الموزع.

Model answer:

Agile methods were designed for small co-located teams communicating informally. Scaling has two facets: scaling up to large systems too big for one small team, and scaling out from specialized teams to widespread use in a large company with many years of experience.

Scrum was designed for co-located teams meeting daily, but much development now involves distributed teams in different places, which gives access to lower cost staff and specialist skills and allows 24-hour development.

Distributed Scrum (Figure 3.10) requires: videoconferencing between product owner and team; the ScrumMaster located with the development team; the Product Owner visiting the developers to build trust; real-time communications such as instant messaging and video calls; continuous integration so everyone knows the product state; and a common development environment for all teams.

📍 المرجع: Lecture 03 — 3.3 and 3.4 Scaling agile methods